```
    if (!MessageDigest.isEqual(computed, provided)) {    #A
        return;    #A
    }

    session.invalidate();    #B
}
```

#A Verify the anti-CSRF token as before.
#B Invalidate the session cookie.

You can now wire up a new logout endpoint. In keeping with our REST-like approach, you can implement logout as a DELETE request to the `/sessions` endpoint. If clients send a DELETE request to `/sessions/xyz`, where `xyz` is the token ID, then the token may be leaked in either the browser history or in server logs. While this may not be a problem for a logout endpoint as the token will be revoked anyway, you should avoid exposing tokens directly in URLs like this. So, in this case, you'll implement logout as a DELETE request to the `/sessions` endpoint (with no token ID in the URL) and the endpoint will retrieve the token ID from the X-CSRF-Token header instead. While there are ways to make this more RESTful, we will keep it simple in this chapter. Listing 4.18 shows the new logout endpoint that retrieves the token ID from the X-CSRF-Token header and then calls the revoke endpoint on the `TokenStore`. Open TokenController.java in your editor and add the new method.

Listing 4.18 The logout endpoint